Name bhavya rana

Sapid 590034689

Batch 15

Algorithm

1. Read the array size `size`.
2. Create the variable-length integer array `arr[size]`.
3. Read all elements into `arr[]`.
4. Read `k`.
5. Pass the address of `arr[]`, `size`, and `k` to `max_subarray_sum()`.
6. In `max_subarray_sum()`:
   - If `k <= 0` or `k >= size`, display `-1`.
   - Calculate the sum of the first `k` elements and store it in `maxSum`.
   - Consider every remaining contiguous subarray of size `k`.
   - Calculate its sum using a nested loop.
   - If the current sum is greater than `maxSum`, update `maxSum`.
7. Display the maximum subarray sum.

Test Case Table

Test	Array	k	Expected Output
1	`2 1 5 1 3 2`	3	`Maximum subarray sum: 9`
2	`2 3 4 1 5`	2	`Maximum subarray sum: 7`
3	`1 2 3`	4	`-1`
4	`1 2 3 4 5`	3	`Maximum subarray sum: 12`
5	`-5 -2 -8 -1`	2	`Maximum subarray sum: -7`
6	`5 10 15`	0	`-1`

C program

```
#include <stdio.h>

void max_subarray_sum(int *arr, int size, int k)
{
```

```
    int i, j;
    int sum, maxSum;

    /* Check whether k is valid */
    if (k <= 0 || k >= size)
    {
        printf("-1\n");
        return;
    }

    /* Calculate sum of first subarray of size k */
    maxSum = 0;
    for (j = 0; j < k; j++)
    {
        maxSum += *(arr + j);
    }

    /* Check all contiguous subarrays */
    for (i = 1; i <= size - k; i++)
    {
        sum = 0;

        for (j = i; j < i + k; j++)
        {
            sum += *(arr + j);
        }

        if (sum > maxSum)
        {
            maxSum = sum;
        }
    }

    printf("Maximum subarray sum: %d\n", maxSum);
}

int main()
{
    int size, i, k;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    int arr[size];
```

```
    printf("Enter %d elements: ", size);

    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    /* Pass array using its address */
    max_subarray_sum(arr, size, k);

    return 0;
}
```

Execution

